Neural Networks and Deep Learning - Lab Assignment 2

Course: CSCI 4922/5922
Student Name: Meghna Nag
Student ID: 111397134

Lab Report: Training Deep Neural Networks on EuroSAT Dataset

1. Introduction

In this lab we focus on how deep models often suffer from training instability and weak gradient flow, especially when using older activation functions like sigmoid. Because of these issues, training deep networks requires more careful choices such as better activation functions, improved optimization methods like mini-batch SGD and momentum, and architectural techniques like skip connections.

2. Methods

Firsly I started with a shallow model to verify dataset difficulty, built a baseline deep model, and then improved training by testing activation functions, mini-batch SGD, momentum, and skip connections.\

2.1 Dataset Selection

The dataset I chose is the ***EuroSAT Land Cover Classification*** dataset (*Link : https://github.com/phelber/EuroSAT*). EuroSAT is a real-world satellite image dataset designed for land-cover classification. Each sample in the dataset is an RGB satellite image, meaning it contains three color channels (Red, Green, Blue).

The end goal is to achieve a classification model where each image falls into one of 10 land-cover categories. These classes represent different types of land environments, such as `forests`, `highways`, `industrial areas`, `rivers` etc. To access the dataset I am using the PyTorch library through `torchvision.datasets.EuroSAT`.

The dataset contains a total of ***27,000 images****, and it includes exactly* ***10 classes****.*

2.2 Finalizing the Dataset (Difficulty Evaluation)

I have trained a shallow two-layer model first, and its accuracy for test set came out as **42.90%.** This makes sence becuse satellite imagery contains complex textures, shapes, and spatial patterns and many land types look visually similar, such as crops versus pasture or residential versus industrial. And hence a two layer mdoel did not turn to be sufficient for this classification problem.

2.3 Dataset Preprocessing

EuroSAT data are raw images and are of different resolutionss hence to use it for a deep learning model we need to a fixed input shape.

I preprocessed the daaset by performing the following operations: resizing, tensor conversion, and normalization.

1: Resizing : Resized all images to `64×64` so the model architecture stays fixed and training remains computationally efficient.

2: Tensor Conversion (ToTensor) : Neural networks cannot directly process raw image objects, so the images needs to be converted into numerical tensors. `ToTensor` converts the image pixels into numerical values that the model can process, and rescales them by dividing each pixel value by 255 so all values fall between 0 and 1.

3: Normalization : Now, the raw pixel values are not centered, which can make training unstable and slow. We normalized each channel using mean 0.5 and standard deviation 0.5, which shifts values roughly into the range [-1, 1]. This improves gradient stability and helps the model learn faster and more consistently.

2.4 Train-Test Split

```
--------------------------------------------------------------------------------
Train class distribution
--------------------------------------------------------------------------------
Total samples in this split: 18900
   0 | AnnualCrop         :  2062  ( 10.91%)
   1 | Forest             :  2127  ( 11.25%)
   2 | HerbaceousVegetation :  2097  ( 11.10%)
   3 | Highway            :  1728  (  9.14%)
   4 | Industrial         :  1770  (  9.37%)
   5 | Pasture            :  1392  (  7.37%)
   6 | PermanentCrop      :  1733  (  9.17%)
   7 | Residential        :  2096  ( 11.09%)
   8 | River              :  1743  (  9.22%)
   9 | SeaLake            :  2152  ( 11.39%)

--------------------------------------------------------------------------------
Test class distribution
--------------------------------------------------------------------------------
Total samples in this split: 8100
   0 | AnnualCrop         :   938  ( 11.58%)
   1 | Forest             :   873  ( 10.78%)
   2 | HerbaceousVegetation :   903  ( 11.15%)
   3 | Highway            :   772  (  9.53%)
   4 | Industrial         :   730  (  9.01%)
   5 | Pasture            :   608  (  7.51%)
   6 | PermanentCrop      :   767  (  9.47%)
   7 | Residential        :   904  ( 11.16%)
   8 | River              :   757  (  9.35%)
   9 | SeaLake            :   848  ( 10.47%)
```

Splited the data using a **70/30**, trining vs testing ratio.

EuroSAT contains 27,000 samples, the split resulted in:

Training set size = 18,900
Test set size = 8,100

Used a seed of 42 so the split would remain consistent across all experiments.

2.5 Model Architectures (Baseline + Extended Models)

```
┌──────────────────────────────────────────┐
│        PHASE 1: PREPROCESSING            │
└──────────────────────────────────────────┘
          Raw RGB Image
                |
                v
          Resize 64×64
                |
                v
To Tensor (3×64×64), scale → [0,1]
                |
                v
Normalize:  X = (X − 0.5) / 0.5
                |
                v
  Final Input Tensor (3×64×64)
```

I designed the multiple neural network architecture with the inputs which are RGB satellite images (64×64). The purpose of the models is to learn visual patterns such as textures, edges, and shape-like structures that distinguish land types.

To make sure there are no discripncies, I used the **same preprocessing**(Resize → ToTensor → Normalize) and the **same fixed split (seed = 42)** across both trianing and testing sets.

2.6 Part 1A: Shallow Two-Layer Model

```
┌──────────────────────────────────────┐
│          PHASE 2: DATA SPLIT         │
└──────────────────────────────────────┘

                27,000 Images
                      |
                      v
            Random Split (seed = 42)
                      |
        ┌─────────────┴─────────────┐
        v                           v
  Train Set (18,900)          Test Set (8,100)
        |                           |
        v                           v
Train Loader (shuffle=True)  Test Loader (shuffle=False)
```

Starting with the shallow two-layer MLP where each input image was flattened.

1. Image shape: (3,64,64)
2. Flattened size: 3 * 64 * 64 = 12,288

Implemented this using the following:

- First fully connected linear layer : fc1: Linear(Input Vector: 12288 → Output vector: 64)
- Second linear layer : fc2: Linear(Hidden Values: 64 → Output(per class value):10)
- Hidden activation: act_sigmoid (output: raw score)
- Output: softmax(logits) (output: probabilities)

I trianed the model with **SGD**, **batch size = 1**, and **learning rate = 1e-3**

2.7 Forward Pass Implementation

```
┌──────────────────────────────────────┐
│      PHASE 3: TRAINING (SGD LOOP)    │
└──────────────────────────────────────┘

For each epoch, for each batch (X, y):
                |
                v
      Forward Pass → logits z
                |
                v
Softmax:  pᵢⱼ = exp(zᵢⱼ) / Σₖ exp(zᵢₖ)
                |
                v
Cross-Entropy:  L = -(1/b) Σ log(pᵢ,yᵢ)
                |
                v
      Backprop:  g = ∇θ L
                |
                v
        Reset grads: θ.grad = 0
                |
                v
Update:
    SGD: θ ← θ − ηg
    Momentum: m←αm+g; θ←θ−ηm
```

I implemented the forward pass using basic tensor operations instead of relying on built-in training utilities. For each layer, the model first applies a linear transformation (weights and bias) using nn.Linear, and then applies a non-linear activation function to introduce learning capacity. I tested three activation functions—**sigmoid**, **tanh**, and **LeakyReLU (slope = 0.1)**—each implemented manually in separate helper functions (act_sigmoid, act_tanh, and act_leaky_relu). Finally, the network outputs raw scores (logits), which I converted into **probabilities across the 10 EuroSAT classes** using a numerically stable softmax implementation (softmax(logits, dim=1)).

2.8 Loss Function: Cross-Entropy (Multi-Class Classification)

Now EuroSAT is a 10-class classification problem, I used **cross-entropy loss** to measure how well the model assigns high probability to the correct class.

After computing class probabilities using **softmax**, I extracted the probability of the true label p_y and computed the loss as $L=-\log(p_y)$, averaged across the mini-batch.

Implemented this in code with function `cross_entropy_from_probs()` using indexing and a mean negative log calculation.

2.9 Training Loop Implementation (SGD Update Rule)

To train the model, I implemented the full training loop manually instead of using PyTorch optimizers. For each mini-batch, I first reset gradients from the previous step, then performed a forward pass to compute predicted class probabilities. Using these probabilities, I computed the cross-entropy loss and called `loss.backward()` to generate gradients for every weight and bias.

After gradients were computed, I updated parameters manually using SGD:

$$\theta_{i+1} = \theta_i - \eta \frac{\partial L}{\partial \theta_i}$$

I accessed parameters through `.weight` and `.bias` using the function `get_named_params()`.

2.10 Part 1B: Baseline Deep Network Design (5+ Layers)

Now implemeting the deeper baseline model. This baseline network was designed to first extract useful spatial features from the satellite images using convolution layers, and then perform classification using multiple fully connected layers.

Model components:

1. A small CNN feature extractor (2 conv layers + pooling)
2. Followed by *5 fully-connected layers* (hidden size 256)
3. And a final output layer for 10 classes

The model begins with a small CNN feature extractor consisting of two convolution layers with max pooling. This reduces the original (64 * 64) image into smaller feature maps while still preserving important patterns such as textures, edges, and land structure. After the second pooling layer, the feature maps have size (32 * 16 * 16), which results in a flattened feature vector of dimension: 32* 16* 16 = 8192

This flattened feature vector is then passed through a deep fully-connected classifier made of five hidden layers (each with 256 units), followed by an output layer producing logits for the 10 EuroSAT classes.

The baseline model used sigmoid activation, SGD training with batch size 1, and a learning rate of 10^(-3).

3. Part 2: Activation Functions and Optimizers

Here I improved the model performance by changing activations and optimization methods.

<u>Defining the Activation Hypothesis</u>

The expectation is both **tanh** and **LeakyReLU** will perform better than sigmoid because sigmoid saturates and weakens gradients in deep stacks. Also between the two, I expected tanh to be stable because it is zero-centered, and I expect LeakyReLU to maintain to have nicer gradients due to its non-zero slope for negative values.

3.1 Part 2A: Baseline Model with tanh Activation

Replaced sigmoid with **tanh** in all hidden layers while keeping the baseline architecture unchanged as per :

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Code implementation: I passed `act_tanh` into the training loop (`run_experiment(... activation=act_tanh ...)`)

Training values : batch size: 1, lr: 1e-3, epochs: 5 (EPOCHS_P2 = 5)

3.2 Part 2B: Baseline Model with LeakyReLU Activation

Next, I replaced sigmoid with LeakyReLU (slope = 0.1):

$$f(x) = \max(x, 0.1x)$$

Code: Activation function: `act_leaky_relu_01()`

Training values: batch size: 1, lr: 1e-3, epochs: 5

3.3 Part 2C: Mini-Batch SGD Implementation

I implemented mini-batch SGD by training the model using batches of images instead of updating the weights after every single image.

As of the code, I used a `DataLoader` and tested three batch sizes: 8, 32, and 64. For each batch, the model computed predictions, calculated the average cross-entropy loss over the batch, and then ran `loss.backward()` to compute gradients.

Since the loss was averaged, the gradients were also effectively averaged across the batch before updating the weights. After comparing results, batch size 8 performed best, so I used it for the next part.

3.4 Part 2D: SGD with Momentum Implementation

Next, I implemented *SGD with momentum* to improve training stability and reduce noisy updates caused by mini-batch gradients.

Now what I did was instead of updating weights only using the current gradient, I stored a running velocity term inside `MomentumState.vel`, which keeps track of previous update directions.

Then, I updated parameters using both the current gradient and the stored momentum term whenever `momentum_alpha > 0`.

The key idea is:

a. If `momentum_alpha = 0`, then there is **no momentum**, and the update becomes normal SGD.
b. While If `momentum_alpha > 0`, then momentum is **enabled**, meaning the new update combines both the current gradient and the previous velocity.

So when `momentum_alpha > 0`, the model does not reset direction every step, instead it keeps moving in a consistent direction, which helps reduce oscillations and speeds up convergence.

I tested momentum values **0.5, 0.9, and 0.95** (using the best activation function and batch size from earlier steps) to find the best training stability and final test performance values.

4. Part 3: Skip Connections

In this part, the motive as to study whether adding skip connections improves training behavior in a deeper network. The main motivation is that very deep networks often suffer from vanishing gradients, where early layers receive very small gradient updates and learn slowly. Skip connections help by creating shortcut paths so information and gradients can flow more easily through the network.

In my implementation, the skip operation was performed as: z = z + states[frm]

4.1 Part 3A: Extended Architecture (+10 Extra Layers, No Skip Connections)

To test deep training behavior, I extended my baseline model by adding **10 extra fully connected layers**. These layers keep the hidden dimension fixed at **256**.

After the CNN feature extractor, the feature maps are flattened into a vector of size: 32 * 16 * 16 = 8192

Then I used a linear projection layer: `fc_in: Linear(8192 → 256)`

After that, I added **10 repeated hidden layers**, each performing: `Linear(256 → 256)` + activation

So each extra layer performs the same computation: $z_{new} = act(W z_{old} + b)$

Implemented these layers with: `extra = nn.ModuleList([nn.Linear(hidden_dim, hidden_dim) for _ in range(extra_layers)])`, where `extra_layers = 10`.

The output of the network is: `fc_out: Linear(256 → 256)`, `out: Linear(256 → 10)` followed by softmax.

4.2 Part 3B: Skip Connection Architecture 1

After training the extended model without skip connections, I implemented skip connections to improve training stability and help gradients flow through the deeper network.

So the output from an earlier hidden layer was reused and added directly into a deeper hidden layer during the forward pass.

To do this in code, I stored the output of each hidden layer inside a list called `states`. Then, when the network reached a layer that had a skip connection, I added the saved activation from the earlier layer to the current activation using residual addition. *This creates shortcut paths through the network and thus reducing the vanishing gradient problem.*

4.3 Part 3C: Skip Connection Architecture 2

For the second part, I tested a different skip pattern. This configuration creates shorter and more frequent skip paths compared to configuration 1. The implementation remained the same, and only the skip configuration list changed.

4.4 Gradient Magnitude Analysis (Vanishing Gradient Evidence)

To actually see the vanishing gradients, I printed the gradient magnitudes during epoch 1 of training for each Part 3 model.

For each parameter, I measured the L1 gradient magnitude, accumulated these values across all updates in epoch 1, averaged them, and then grouped weight and bias gradients by layer to compute a layer-wise gradient report. Finally, I reported the top 15 layers with the highest average gradient magnitude.

5. Results

Table 1: Final Test Results Across Experiments

Experiment	Activation	Batch	LR	Alpha	Beta	Acc	MacroP	MacroR
Shallow MLP	sigmoid	1	1.0e-03	0.00	0	42.90%	39.94%	41.42%
Baseline Deep Net	sigmoid	1	1.0e-03	0.00	0	11.58%	1.16%	10.00%
Baseline + tanh	tanh	1	1.0e-03	0.00	0	75.33%	74.05%	75.05%
Baseline + LeakyReLU	leaky_relu(0.1)	1	1.0e-03	0.00	0	60.10%	58.14%	59.68%
Mini-batch SGD b=8	tanh	8	1.0e-03	0.00	0	28.99%	14.64%	28.74%
Mini-batch SGD b=32	tanh	32	1.0e-03	0.00	0	20.91%	11.76%	20.78%
Mini-batch SGD b=64	tanh	64	1.0e-03	0.00	0	14.59%	6.07%	13.60%
Momentum alpha=0.5	tanh	8	1.0e-03	0.50	0	56.70%	56.14%	55.69%
Momentum alpha=0.9	tanh	8	1.0e-03	0.90	0	73.63%	77.05%	72.93%
Momentum alpha=0.95	tanh	8	1.0e-03	0.95	0	73.99%	77.16%	73.88%
Extended (+10), no skips	tanh	8	1.0e-03	0.95	0	64.17%	63.21%	64.34%
Skip config 1	tanh	8	1.0e-03	0.95	0	80.27%	80.05%	80.08%
Skip config 2	tanh	8	1.0e-03	0.95	0	81.00%	80.73%	80.64%

Table 1: *Summary of all EuroSAT experiments showing the effect of activation functions, batch size, momentum, extended depth, and skip connections on test Accuracy, Macro Precision, and Macro Recall.*

Discussing results across all the experiments:

1. Difficulty check (shallow model)

The shallow 2-layer MLP (sigmoid, SGD, batch=1) reached **42.90% accuracy** hence showing EuroSAT is not trivially solved by a shallow model and is appropriate for deeper-model experiments.

2. Baseline deep model (sigmoid)

The baseline deep network using **sigmoid** performed extremely poorly, reaching only **11.58% accuracy**, which is essentially near random chance (10% for 10 classes).

3. Activation function experiment

While replacing sigmoid with **tanh** produced a major improvement, reaching **75.33% accuracy**.

And with **LeakyReLU** improved performance over sigmoid but was clearly worse than tanh, reaching **60.10%**.

Because **tanh** had the highest accuracy, I selected that for the remaining experiments.

4. Mini-batch SGD

Using tanh, I tested mini-batch SGD with batch sizes **8, 32, 64**. Performance dropped sharply compared to batch=1.

Among these, **batch size 8** performed best with 28.99% accuracy, so used it for momentum and Part 3.

5. SGD with momentum

Adding momentum greatly improved mini-batch training (batch=8, tanh) and α=0.95: 73.99% accuracy (performed best)

The best momentum setting was α=0.95, which was used for Part 3.

6. Skip connections

I extended the model by adding *10 extra fully connected layers* (no skips first), then added skip connections.

i) Part 3A (no skips)**:** 64.17% accuracy

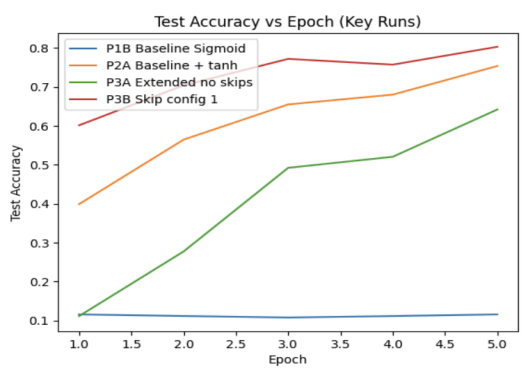
ii) Part 3B (skip config 1)**:** 80.27% accuracy

iii) Part 3C (skip config 2): 81.00% accuracy (best overall)

Skip connections produced the strongest final results, improving accuracy to **81.00%**, and also improved Macro Precision/Recall to **80%**.

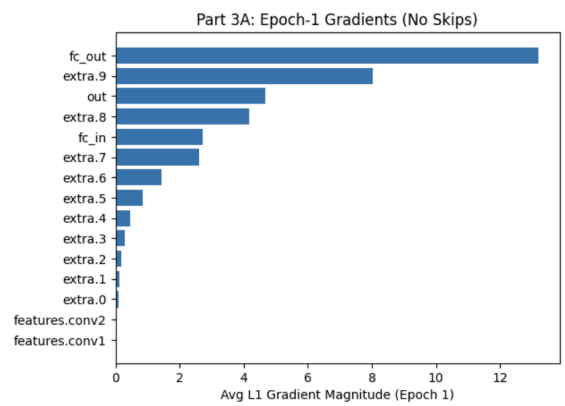
6. Plots:

Chart 1: Test Accuracy vs Epoch (Key Runs)



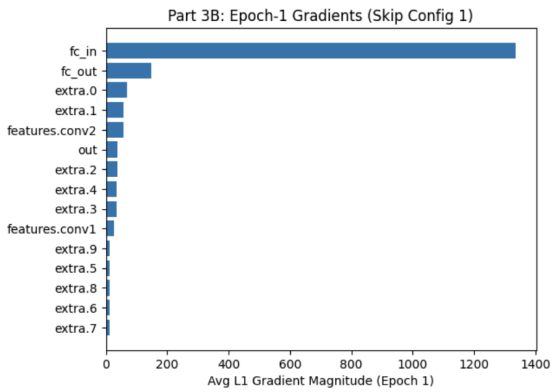
Shows how test accuracy changes over training epochs for important models, making it easy to compare which architecture learns faster and reaches higher accuracy.

Chart 2: Part 3A Epoch-1 Gradients (No Skips)



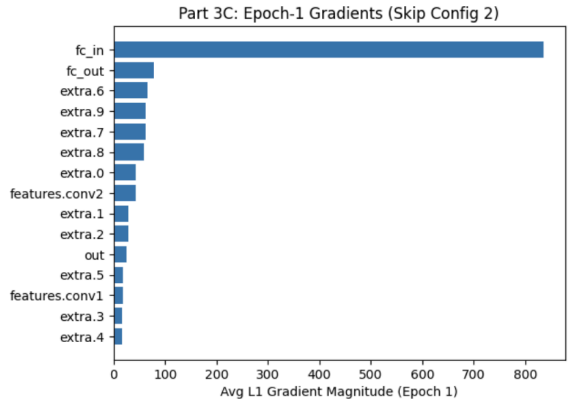
Shows gradient magnitudes across layers in the deep model without skip connections, highlighting vanishing gradients in early layers.

Chart 3: Part 3B Epoch-1 Gradients (Skip Config 1)



Shows how skip connections dramatically increase gradient flow across layers compared to the no-skip model.

Chart 4: Part 3C Epoch-1 Gradients (Skip Config 2)



Shows gradient magnitudes with a different skip pattern, allowing comparison of which skip design improves gradient propagation better.

7. Analysis

Across the experiments, the biggest takeaway was that this EuroSAT task is *not easy classification*, and deep networks only trained well when the optimization setup allowed gradients to reliably reach early layers.

We have seen that the shallow MLP reached 42.90% accuracy also, the deep sigmoid baseline (Part 1B) collapsed to 11.58%, only slightly above random chance for 10 classes, this gap strongly suggests the problem was not model capacity, but *trainability*. The **sigmoid deep baseline failed due to vanishing gradients**.

Sigmoid activation constricts every layer's output into a very small range (between 0 and 1). When many sigmoid layers are stacked, the model's learning signal becomes weaker and weaker as it moves backward during training. Because of this, the first layers (especially the CNN feature extractor) barely get updated, so they do not learn useful image features.

This is supported by the gradient report from the deep model without skip connections (Part 3A). The first convolution layer (`features.conv1`) had an extremely small gradient (around 0.003847), while a later layer like `fc_out` had a much larger gradient (around 13.19). *This shows that most learning happened only in the later layers, while the earlier layers stayed almost unchanged*. That is why the deep sigmoid model looked stuck even after training for several epochs.

The biggest improvement came from changing the activation function. Switching sigmoid to **tanh** increased accuracy from **11.58% to 75.33%**, which is a huge jump. This happened because tanh keeps outputs centered around zero and allows stronger gradient flow, so the model can update early layers more effectively. LeakyReLU also improved results compared to sigmoid (60.10%), but it still performed worse than tanh in this setup.

Another trend was that mini-batch SGD performed worse when the learning rate and epochs were kept fixed. As batch size increased, accuracy dropped from *28.99% (b=8) to 20.91% (b=32) and 14.59% (b=64)*. This happened because larger batches produce fewer weight updates per epoch, so the model learns more slowly. With only 5 epochs and no learning-rate adjustment, the model did not get enough chances to improve.

Momentum helped fix this issue. With batch size 8 and tanh, adding momentum improved accuracy from **28.99% to 73.99%** (best at α=0.95). Momentum works by remembering the previous update direction, so the model does not change direction randomly every step. This makes training smoother and faster, especially when mini-batch updates are less frequent.

The best overall performance came from skip connections. The extended deep model without skips reached 64.17%, but adding skips improved accuracy to 80.27% (config 1) and 81.00% (config 2). Skip connections help because they allow information and gradients to move through the network more easily, instead of being forced to pass through every single layer. The gradient plots confirm this: ***without skips, early layers had tiny gradients, but with skips, early-layer gradients became much larger, meaning those layers finally learned properly.***

Overall, **tanh** gave the largest single improvement, **skip connections** produced the best final model, and **momentum** was essential for making mini-batch training work. ***These results show that better learning performance came mainly from improving gradient flow, not just adding more layers***.

Discussing the graphs :

Chart 1: Test Accuracy vs Epoch (Key Runs)

The chart shows that the sigmoid deep baseline fails to learn beyond random chance, in contrast, the tanh baseline improves rapidly and reaches around 75% by epoch 5. The skip-connection model improves the fastest and achieves the best final accuracy (~80%), outperforming the deeper no-skip model.

Chart 2: Epoch-1 Gradients (No Skips)

The no-skip deep model shows very large gradients in later layers like fc_out, but extremely small gradients in early layers such as features.conv1 and features.conv2. This uneven pattern indicates that learning signals are not reaching the early layers properly. Overall, this is clear evidence of the vanishing gradient problem.

Chart 3: Gradients (Skip Config 1)

With skip connections enabled, gradient magnitudes increase dramatically across the network, especially in early layers like fc_in. Compared to the no-skip model, early CNN feature layers also receive much stronger gradients. This shows skip connections significantly improve gradient flow and allow deeper networks to train more effectively.

Chart 4: Epoch-1 Gradients (Skip Config 2)

This also produces much stronger gradients than the no-skip model, confirming that skip connections reduce vanishing gradients. The gradient values are spread more evenly across layers compared to skip config 1. This improved training stability matches the slightly better final accuracy (81%) achieved by this configuration.

8. Conclusion

From the observation we can safely say that deep model performance on EuroSAT is strongly influenced by gradient stability and optimization design. The deep sigmoid baseline failed to learn and stayed near random accuracy, indicating that sigmoid-based deep stacks suffer from weak learning signals in early layers. Switching the activation function to **tanh** produced the largest single improvement and allowed the network to train effectively, confirming that activation choice is critical for deep architectures.

Mini-batch SGD performed poorly under the fixed learning rate and limited epoch budget, likely due to fewer parameter updates and slower convergence. However, adding **momentum** significantly improved mini-batch training by stabilizing update directions and accelerating learning. The best results came from **skip connections**, which produced the highest accuracy and also showed much larger gradient magnitudes in early layers. Overall, the results confirm that deeper networks only provide benefits when training is supported by effective activations, optimizers, and architectural mechanisms like residual connections.

9. Appendix:

```
"""
==============================================================
LAB 2 (CSCI 5922) — EUROSAT LAND-COVER CLASSIFICATION
==============================================================
"""

import math
from dataclasses import dataclass
from typing import Callable, Dict, List, Tuple, Optional

import torch
from torch import nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

import matplotlib.pyplot as plt


# ============================================================
# 0) Pretty printing helpers (for report-ready output)
# ============================================================
def banner(title: str) -> None:
  print("\n" + "=" * 78)
  print(title)
  print("=" * 78)

def subbanner(title: str) -> None:
  print("\n" + "-" * 78)
  print(title)
  print("-" * 78)

def fmt_pct(x: float) -> str:
  return f"{100.0 * x:6.2f}%"

def fmt_float(x: float) -> str:
  return f"{x:.6f}"
```

```python
def print_kv(items: List[Tuple[str, str]]) -> None:
    maxk = max(len(k) for k, _ in items) if items else 0
    for k, v in items:
        print(f"{k:<{maxk}} : {v}")

def random_chance_accuracy(num_classes: int) -> float:
    return 1.0 / max(1, num_classes)

def difficulty_pass_fail(acc: float, threshold: float = 0.50) -> str:
    return "PASS (dataset is difficult enough)" if acc <= threshold else "FAIL (too easy)"


# ============================================================
# 1) Reproducibility (so results are consistent)
# ============================================================
def set_seed(seed: int = 42) -> None:
    """
    Why we do this:
    - Train/test split uses randomness.
    - Weight initialization uses randomness.
    - Without a fixed seed, two runs might differ and comparisons become unfair.
    """
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
set_seed(42)


# ============================================================
# 2) Manual activation functions (forward pass from scratch)
# ============================================================
def act_sigmoid(x: torch.Tensor) -> torch.Tensor:
    """Sigmoid: σ(x)=1/(1+e^{-x}). Can cause vanishing gradients in deep nets."""
    return 1.0 / (1.0 + torch.exp(-x))

def act_tanh(x: torch.Tensor) -> torch.Tensor:
    """tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x}). Zero-centered; often more stable."""
    ex = torch.exp(x)
```

```
    enx = torch.exp(-x)
    return (ex - enx) / (ex + enx)

def act_leaky_relu(x: torch.Tensor, slope: float = 0.1) -> torch.Tensor:
    """LeakyReLU: f(x)=x if x>=0 else slope*x. Keeps gradient for negative inputs."""
    return torch.maximum(x, slope * x)

def act_leaky_relu_01(x: torch.Tensor) -> torch.Tensor:
    """Helper wrapper so we don't use lambdas in experiments."""
    return act_leaky_relu(x, slope=0.1)


# ============================================================
# 3) Softmax + Cross-Entropy from scratch
# ============================================================
def softmax(logits: torch.Tensor, dim: int = 1) -> torch.Tensor:
    """
    Stable softmax:
      softmax(z)_i = exp(z_i - max(z)) / sum_j exp(z_j - max(z))
    """
    maxv = torch.max(logits, dim=dim, keepdim=True).values
    ex = torch.exp(logits - maxv)
    return ex / torch.sum(ex, dim=dim, keepdim=True)

def cross_entropy_from_probs(probs: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """
    Multi-class NLL from softmax probabilities:
      L = mean_i [-log(p_i[y_i])]
    """
    eps = 1e-12
    picked = probs[torch.arange(probs.size(0)), y]
    return -torch.mean(torch.log(picked + eps))


# ============================================================
# 4) Metrics: Accuracy, Macro Precision, Macro Recall (required)
# ============================================================
@torch.no_grad()
def compute_metrics(y_true: torch.Tensor, y_pred: torch.Tensor, num_classes: int) -> Dict[str, float]:
```

```
    cm = torch.zeros((num_classes, num_classes), dtype=torch.int64)
    for t, p in zip(y_true, y_pred):
        cm[t.long(), p.long()] += 1

    total = cm.sum().item()
    correct = torch.trace(cm).item()
    acc = correct / total if total > 0 else 0.0

    precisions, recalls = [], []
    for c in range(num_classes):
        tp = cm[c, c].item()
        fp = cm[:, c].sum().item() - tp
        fn = cm[c, :].sum().item() - tp
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        rec  = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        precisions.append(prec)
        recalls.append(rec)

    return {
        "accuracy": acc,
        "macro_precision": sum(precisions) / max(1, num_classes),
        "macro_recall": sum(recalls) / max(1, num_classes),
    }

@torch.no_grad()
def evaluate(
    model: nn.Module,
    loader: DataLoader,
    num_classes: int,
    activation: Callable[[torch.Tensor], torch.Tensor],
    skip_config: Optional[List[Tuple[int, int]]] = None
) -> Dict[str, float]:
    model.eval()
    y_true_all, y_pred_all = [], []

    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        probs = forward_probs(model, xb, activation, skip_config)
        pred = torch.argmax(probs, dim=1)
```

```
        y_true_all.append(yb.detach().cpu())
        y_pred_all.append(pred.detach().cpu())

    y_true = torch.cat(y_true_all, dim=0)
    y_pred = torch.cat(y_pred_all, dim=0)
    return compute_metrics(y_true, y_pred, num_classes)


# ============================================================
# 5) Dataset: EuroSAT + ONE fixed split for all experiments
# ============================================================
def build_eurosat_dataset(root="./data", image_size=64):
    """
    Preprocessing:
    - Resize to fixed (image_size, image_size)
    - ToTensor maps to [0,1]
    - Normalize with mean=0.5/std=0.5 -> approximately [-1,1]
    """
    tfm = transforms.Compose([
        transforms.Resize((image_size, image_size)),
        transforms.ToTensor(),
        transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5)),
    ])
    ds = datasets.EuroSAT(root=root, download=True, transform=tfm)
    return ds

def fixed_split(ds, train_ratio=0.7, seed=42):
    n_total = len(ds)
    n_train = int(train_ratio * n_total)
    n_test = n_total - n_train
    gen = torch.Generator().manual_seed(seed)
    train_ds, test_ds = random_split(ds, [n_train, n_test], generator=gen)
    return train_ds, test_ds

def make_loader(subset, batch_size, shuffle, num_workers=2):
    return DataLoader(
        subset,
        batch_size=batch_size,
        shuffle=shuffle,
```

```
        num_workers=num_workers,
        pin_memory=True
    )

def dataset_quick_stats(train_ds, test_ds, num_classes: int) -> Dict[str, Dict[int, int]]:
    """
    Small helper to show class distribution in train/test.
    NOTE: We only use dataset indexing here; no training logic depends on it.
    """
    train_counts = {i: 0 for i in range(num_classes)}
    test_counts = {i: 0 for i in range(num_classes)}

    # random_split returns Subset objects with .indices into original dataset
    for idx in train_ds.indices:
        _, y = train_ds.dataset[idx]
        train_counts[int(y)] += 1

    for idx in test_ds.indices:
        _, y = test_ds.dataset[idx]
        test_counts[int(y)] += 1

    return {"train": train_counts, "test": test_counts}

def print_class_counts(ds_classes: List[str], counts: Dict[int, int], title: str) -> None:
    subbanner(title)
    total = sum(counts.values())
    print(f"Total samples in this split: {total}")
    for i, name in enumerate(ds_classes):
        c = counts.get(i, 0)
        frac = (c / total) if total > 0 else 0.0
        print(f"  {i:2d} | {name:<18s} : {c:5d}  ({fmt_pct(frac)})")


# ============================================================
# 6) Models: only allowed modules + manual activations
# ============================================================
class SimpleCNNFeature(nn.Module):
    """
    conv -> activation -> pool -> conv -> activation -> pool
```

```
    """
    def __init__(self, in_ch=3):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, 16, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(2)

    def forward(self, x: torch.Tensor, activation: Callable[[torch.Tensor], torch.Tensor]) -> torch.Tensor:
        x = self.pool1(activation(self.conv1(x)))
        x = self.pool2(activation(self.conv2(x)))
        return x

class ShallowTwoLayerNet(nn.Module):
    """
    Part 1A difficulty check:
      Flatten -> Linear -> sigmoid -> Linear -> softmax
    """
    def __init__(self, in_dim: int, hidden_dim: int, num_classes: int):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x.view(x.size(0), -1)
        h = act_sigmoid(self.fc1(x))
        logits = self.fc2(h)
        return softmax(logits, dim=1)

class BaselineDeepNet(nn.Module):
    """
    Part 1B baseline deep model (>=5 parameterized layers).
    """
    def __init__(self, image_size: int, num_classes: int, hidden_dim: int = 256):
        super().__init__()
        self.features = SimpleCNNFeature(3)

        feat_h = image_size // 4
        feat_w = image_size // 4
```

```
        feat_dim = 32 * feat_h * feat_w

        self.fc1 = nn.Linear(feat_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, hidden_dim)
        self.fc4 = nn.Linear(hidden_dim, hidden_dim)
        self.fc5 = nn.Linear(hidden_dim, hidden_dim)
        self.out = nn.Linear(hidden_dim, num_classes)

    def forward(self, x: torch.Tensor, activation: Callable[[torch.Tensor], torch.Tensor]) -> torch.Tensor:
        z = self.features(x, activation)
        z = z.view(z.size(0), -1)

        z = activation(self.fc1(z))
        z = activation(self.fc2(z))
        z = activation(self.fc3(z))
        z = activation(self.fc4(z))
        z = activation(self.fc5(z))

        logits = self.out(z)
        return softmax(logits, dim=1)

class ExtendedDeepNetWithSkips(nn.Module):
    """
    Part 3: Extended model (+10 layers) + optional skip connections.
    """
    def __init__(self, image_size: int, num_classes: int, hidden_dim: int = 256, extra_layers: int = 10):
        super().__init__()
        self.features = SimpleCNNFeature(3)

        feat_h = image_size // 4
        feat_w = image_size // 4
        feat_dim = 32 * feat_h * feat_w

        self.fc_in = nn.Linear(feat_dim, hidden_dim)
        self.extra = nn.ModuleList([nn.Linear(hidden_dim, hidden_dim) for _ in range(extra_layers)])
        self.fc_out = nn.Linear(hidden_dim, hidden_dim)
        self.out = nn.Linear(hidden_dim, num_classes)
```

```
    def forward(
        self,
        x: torch.Tensor,
        activation: Callable[[torch.Tensor], torch.Tensor],
        skip_config: Optional[List[Tuple[int, int]]] = None
    ) -> torch.Tensor:
        z = self.features(x, activation)
        z = z.view(z.size(0), -1)

        states: List[torch.Tensor] = []
        z = activation(self.fc_in(z))
        states.append(z)  # state 0

        skip_to: Dict[int, List[int]] = {}
        if skip_config:
            for frm, to in skip_config:
                skip_to.setdefault(to, []).append(frm)

        for i, layer in enumerate(self.extra, start=1):
            z = activation(layer(z))
            if i in skip_to:
                for frm in skip_to[i]:
                    z = z + states[frm]  # real in-stream residual
            states.append(z)

        z = activation(self.fc_out(z))
        logits = self.out(z)
        return softmax(logits, dim=1)


# ============================================================
# 7) STRICT RULE COMPLIANCE:
#    NO model.parameters() and NO model.named_parameters()
#    Collect params ONLY via module.weight / module.bias
# ============================================================
def _add_param(named: List[Tuple[str, torch.Tensor]], name: str, t: Optional[torch.Tensor]) -> None:
    if t is not None:
        named.append((name, t))
```

```python
def get_named_params(model: nn.Module) -> List[Tuple[str, torch.Tensor]]:
    named: List[Tuple[str, torch.Tensor]] = []

    if isinstance(model, ShallowTwoLayerNet):
        _add_param(named, "fc1.weight", model.fc1.weight)
        _add_param(named, "fc1.bias",   model.fc1.bias)
        _add_param(named, "fc2.weight", model.fc2.weight)
        _add_param(named, "fc2.bias",   model.fc2.bias)
        return named

    if isinstance(model, BaselineDeepNet):
        _add_param(named, "features.conv1.weight", model.features.conv1.weight)
        _add_param(named, "features.conv1.bias",   model.features.conv1.bias)
        _add_param(named, "features.conv2.weight", model.features.conv2.weight)
        _add_param(named, "features.conv2.bias",   model.features.conv2.bias)

        for i in range(1, 6):
            fc = getattr(model, f"fc{i}")
            _add_param(named, f"fc{i}.weight", fc.weight)
            _add_param(named, f"fc{i}.bias",   fc.bias)

        _add_param(named, "out.weight", model.out.weight)
        _add_param(named, "out.bias",   model.out.bias)
        return named

    if isinstance(model, ExtendedDeepNetWithSkips):
        _add_param(named, "features.conv1.weight", model.features.conv1.weight)
        _add_param(named, "features.conv1.bias",   model.features.conv1.bias)
        _add_param(named, "features.conv2.weight", model.features.conv2.weight)
        _add_param(named, "features.conv2.bias",   model.features.conv2.bias)

        _add_param(named, "fc_in.weight", model.fc_in.weight)
        _add_param(named, "fc_in.bias",   model.fc_in.bias)

        for i, layer in enumerate(model.extra):
            _add_param(named, f"extra.{i}.weight", layer.weight)
            _add_param(named, f"extra.{i}.bias",   layer.bias)

        _add_param(named, "fc_out.weight", model.fc_out.weight)
```

```
        _add_param(named, "fc_out.bias",   model.fc_out.bias)

        _add_param(named, "out.weight", model.out.weight)
        _add_param(named, "out.bias",   model.out.bias)
        return named

    raise ValueError("Unknown model type")

def zero_grads_named(named_params: List[Tuple[str, torch.Tensor]]) -> None:
    for _, p in named_params:
        p.grad = None


# ============================================================
# 8) Manual optimizers: SGD / Momentum / Weight Decay
# ============================================================
@dataclass
class OptimConfig:
    lr: float = 1e-3
    momentum_alpha: float = 0.0
    weight_decay_beta: float = 0.0
    batch_size: int = 1

class MomentumState:
    def __init__(self):
        self.vel: Dict[int, torch.Tensor] = {}

def apply_update(named_params: List[Tuple[str, torch.Tensor]], cfg: OptimConfig, mom:
MomentumState) -> None:
    for _, p in named_params:
        if p.grad is None:
            continue
        g = p.grad.data

        if cfg.momentum_alpha > 0.0 or cfg.weight_decay_beta > 0.0:
            key = id(p)
            if key not in mom.vel:
                mom.vel[key] = torch.zeros_like(p.data)
```

```
            v = mom.vel[key]
            if cfg.weight_decay_beta != 0.0:
                v_next = cfg.momentum_alpha * v + g + cfg.weight_decay_beta * p.data
            else:
                v_next = cfg.momentum_alpha * v + g

            mom.vel[key] = v_next
            p.data = p.data - cfg.lr * v_next
        else:
            p.data = p.data - cfg.lr * g


# ============================================================
# 9) Part 3 Gradient reporting: per-layer L1 + overall avg
# ============================================================
def layer_key_from_param_name(param_name: str) -> str:
    if param_name.endswith(".weight"):
        return param_name[:-len(".weight")]
    if param_name.endswith(".bias"):
        return param_name[:-len(".bias")]
    return param_name

def summarize_epoch1_gradients_layerwise(grad_l1_param_avg: Dict[str, float]) -> Dict[str, object]:
    buckets: Dict[str, List[float]] = {}
    for pname, val in grad_l1_param_avg.items():
        lk = layer_key_from_param_name(pname)
        buckets.setdefault(lk, []).append(val)

    layer_avg = {lk: sum(vals) / len(vals) for lk, vals in buckets.items()}
    overall_layer_avg = sum(layer_avg.values()) / max(1, len(layer_avg))
    return {"layer_avg": layer_avg, "overall_layer_avg": overall_layer_avg}

def print_layerwise_grad_report(title: str, grad_summary: Dict[str, object], top_k: int = 15) -> None:
    subbanner(title)
    layer_avg: Dict[str, float] = grad_summary["layer_avg"]
    overall = grad_summary["overall_layer_avg"]

    items = sorted(layer_avg.items(), key=lambda kv: kv[1], reverse=True)
```

```
    print(f"Overall avg across layers (epoch 1): {fmt_float(overall)}")
    print(f"Top {top_k} layers by avg L1 grad size (epoch 1):")
    for lk, v in items[:top_k]:
        print(f"  {lk:<28s} : {fmt_float(v)}")


# ============================================================
# 10) Forward dispatcher
# ============================================================
def forward_probs(
    model: nn.Module,
    xb: torch.Tensor,
    activation: Callable[[torch.Tensor], torch.Tensor],
    skip_config: Optional[List[Tuple[int, int]]] = None
) -> torch.Tensor:
    if isinstance(model, ShallowTwoLayerNet):
        return model(xb)
    if isinstance(model, BaselineDeepNet):
        return model(xb, activation)
    if isinstance(model, ExtendedDeepNetWithSkips):
        return model(xb, activation, skip_config)
    raise ValueError("Unknown model class")


# ============================================================
# 11) Training loop (mini-batch SGD via mean loss)
# ============================================================
def train_one_epoch(
    model: nn.Module,
    loader: DataLoader,
    activation: Callable[[torch.Tensor], torch.Tensor],
    cfg: OptimConfig,
    mom_state: MomentumState,
    record_grad_epoch1: bool = False,
    skip_config: Optional[List[Tuple[int, int]]] = None
) -> Tuple[float, Optional[Dict[str, float]]]:

    model.train()
    named_params = get_named_params(model)
```

```
    total_loss = 0.0
    num_updates = 0
    grad_l1_sum = {} if record_grad_epoch1 else None

    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)

        # (1) reset gradients
        zero_grads_named(named_params)

        # (2) forward
        probs = forward_probs(model, xb, activation, skip_config)

        # (3) mean loss over batch
        loss = cross_entropy_from_probs(probs, yb)

        # (4) backward -> grads of mean loss = mean gradient across batch samples
        loss.backward()

        # (5) record epoch-1 gradients if required
        if record_grad_epoch1:
            for pname, p in named_params:
                if p.grad is not None:
                    grad_l1_sum[pname] = grad_l1_sum.get(pname, 0.0) +
torch.sum(torch.abs(p.grad)).item()

        # (6) update
        apply_update(named_params, cfg, mom_state)

        total_loss += loss.item()
        num_updates += 1

    avg_loss = total_loss / max(1, num_updates)

    if record_grad_epoch1 and grad_l1_sum is not None:
        grad_l1_avg = {k: v / max(1, num_updates) for k, v in grad_l1_sum.items()}
        return avg_loss, grad_l1_avg
```

```
    return avg_loss, None


def run_experiment(
    deliverable_tag: str,
    title: str,
    model: nn.Module,
    train_loader: DataLoader,
    test_loader: DataLoader,
    num_classes: int,
    activation: Callable[[torch.Tensor], torch.Tensor],
    cfg: OptimConfig,
    epochs: int,
    record_grad_epoch1: bool = False,
    skip_config: Optional[List[Tuple[int, int]]] = None,
    history: Optional[Dict[str, List[float]]] = None
) -> Dict[str, object]:

    banner(f"{deliverable_tag} — {title}")

    # Print configuration in a human way
    print_kv([
        ("Epochs", str(epochs)),
        ("Batch size", str(cfg.batch_size)),
        ("Learning rate", str(cfg.lr)),
        ("Momentum alpha", str(cfg.momentum_alpha)),
        ("Weight decay beta", str(cfg.weight_decay_beta)),
    ])

    model = model.to(DEVICE)
    mom_state = MomentumState()

    epoch1_grad_param_avg = None
    last_metrics = None

    for ep in range(1, epochs + 1):
        record_now = (record_grad_epoch1 and ep == 1)

        train_loss, grad_param_avg = train_one_epoch(
```

```
        model=model,
        loader=train_loader,
        activation=activation,
        cfg=cfg,
        mom_state=mom_state,
        record_grad_epoch1=record_now,
        skip_config=skip_config
    )

    metrics = evaluate(model, test_loader, num_classes, activation, skip_config)
    last_metrics = metrics

    if record_now:
        epoch1_grad_param_avg = grad_param_avg

    if history is not None:
        history["train_loss"].append(train_loss)
        history["test_acc"].append(metrics["accuracy"])

    print(
        f"Epoch {ep:02d} | "
        f"train_loss={train_loss:.4f} | "
        f"test_acc={fmt_pct(metrics['accuracy'])} | "
        f"macroP={fmt_pct(metrics['macro_precision'])} | "
        f"macroR={fmt_pct(metrics['macro_recall'])}"
    )

grad_layer_report = None
if record_grad_epoch1 and epoch1_grad_param_avg is not None:
    grad_layer_report = summarize_epoch1_gradients_layerwise(epoch1_grad_param_avg)

# Friendly final summary line (report-ready)
subbanner("Final TEST Metrics (use this in Results)")
print_kv([
    ("Accuracy", fmt_pct(last_metrics["accuracy"])),
    ("Macro Precision", fmt_pct(last_metrics["macro_precision"])),
    ("Macro Recall", fmt_pct(last_metrics["macro_recall"])),
])
```

```
    return {
        "deliverable": deliverable_tag,
        "title": title,
        "final_metrics": last_metrics,
        "epoch1_grad_param_avg": epoch1_grad_param_avg,
        "epoch1_grad_layer_report": grad_layer_report,
        "model": model,
        "cfg": cfg,
    }


# ============================================================
# 12) Results table
# ============================================================
def add_result_row(
    results: List[Dict[str, object]],
    deliverable: str,
    experiment: str,
    activation_name: str,
    batch_size: int,
    lr: float,
    momentum_alpha: float,
    weight_decay_beta: float,
    metrics: Dict[str, float]
) -> None:
    results.append({
        "Deliverable": deliverable,
        "Experiment": experiment,
        "Activation": activation_name,
        "Batch": batch_size,
        "LR": lr,
        "Alpha": momentum_alpha,
        "Beta": weight_decay_beta,
        "Acc": metrics["accuracy"],
        "MacroP": metrics["macro_precision"],
        "MacroR": metrics["macro_recall"],
    })

def print_results_summary_table(results: List[Dict[str, object]]) -> None:
```

```
    banner("RESULTS SUMMARY TABLE")

    headers = ["Deliverable", "Experiment", "Activation", "Batch", "LR", "Alpha", "Beta", "Acc",
"MacroP", "MacroR"]
    rows = []
    for r in results:
        rows.append([
            r["Deliverable"],
            r["Experiment"],
            r["Activation"],
            str(r["Batch"]),
            f"{r['LR']:.1e}",
            f"{r['Alpha']:.2f}",
            f"{r['Beta']:.1e}" if r["Beta"] != 0 else "0",
            fmt_pct(r["Acc"]),
            fmt_pct(r["MacroP"]),
            fmt_pct(r["MacroR"]),
        ])

    # Compute column widths
    widths = [len(h) for h in headers]
    for row in rows:
        for j, cell in enumerate(row):
            widths[j] = max(widths[j], len(cell))

    # Print table
    header_line = " | ".join(headers[j].ljust(widths[j]) for j in range(len(headers)))
    sep_line = "-+-".join("-" * widths[j] for j in range(len(headers)))
    print(header_line)
    print(sep_line)
    for row in rows:
        print(" | ".join(row[j].ljust(widths[j]) for j in range(len(headers))))


# ============================================================
# 13) Plot helpers (optional)
# ============================================================
def plot_learning_curves(histories: Dict[str, Dict[str, List[float]]]) -> None:
    plt.figure()
```

```
    for name, h in histories.items():
        plt.plot(range(1, len(h["test_acc"]) + 1), h["test_acc"], label=name)
    plt.xlabel("Epoch")
    plt.ylabel("Test Accuracy")
    plt.title("Test Accuracy vs Epoch (Key Runs)")
    plt.legend(loc="upper left", bbox_to_anchor=(1.02, 1), fontsize=9)
    plt.tight_layout()
    plt.show()

def plot_top_gradients(title: str, grad_layer_report: Dict[str, object], top_k: int = 15) -> None:
    layer_avg: Dict[str, float] = grad_layer_report["layer_avg"]
    items = sorted(layer_avg.items(), key=lambda kv: kv[1], reverse=True)[:top_k]

    labels = [k for k, _ in items][::-1]
    vals = [v for _, v in items][::-1]

    plt.figure()
    plt.barh(labels, vals)
    plt.xlabel("Avg L1 Gradient Magnitude (Epoch 1)")
    plt.title(title)
    plt.show()


# ============================================================
# 14) MAIN: Parts 1–3 with deliverable-based printing
# ============================================================
def main():
    banner("LAB 2 START — EUROSTAT LAND COVER CLASSIFICATION")
    print_kv([("Device detected", DEVICE), ("Seed", "42")])

    IMAGE_SIZE = 64
    EPOCHS_P1A = 3
    EPOCHS_P1B = 5
    EPOCHS_P2  = 5
    EPOCHS_P3  = 5

    # ---------------------------
    # DELIVERABLE: Dataset + Preprocessing + Split
    # ---------------------------
```

```python
banner("DELIVERABLE (Part 1): Dataset Selection + Preprocessing + Fixed Split")

ds = build_eurosat_dataset(root="./data", image_size=IMAGE_SIZE)
num_classes = len(ds.classes)

print_kv([
    ("Dataset", "torchvision.datasets.EuroSAT"),
    ("Classes", str(num_classes)),
    ("Total samples", str(len(ds))),
    ("Image size", f"{IMAGE_SIZE}x{IMAGE_SIZE}"),
    ("Transforms", "Resize -> ToTensor -> Normalize(mean=0.5,std=0.5)"),
    ("Split", "70% train / 30% test (fixed seed split)"),
])

train_ds, test_ds = fixed_split(ds, train_ratio=0.7, seed=42)
print_kv([
    ("Train size", str(len(train_ds))),
    ("Test size", str(len(test_ds))),
])

# Optional: class distribution print (helps Methods justification)
counts = dataset_quick_stats(train_ds, test_ds, num_classes)
print_class_counts(ds.classes, counts["train"], "Train class distribution")
print_class_counts(ds.classes, counts["test"], "Test class distribution")

def loaders_for_b(b: int):
    return (
        make_loader(train_ds, batch_size=b, shuffle=True, num_workers=2),
        make_loader(test_ds, batch_size=b, shuffle=False, num_workers=2),
    )

# Store results + histories for plots
results: List[Dict[str, object]] = []
histories: Dict[str, Dict[str, List[float]]] = {}

# ========================================================
# PART 1A — Difficulty Check (Shallow 2-layer Sigmoid)
# ========================================================
train_b1, test_b1 = loaders_for_b(1)
```

```
shallow_in_dim = 3 * IMAGE_SIZE * IMAGE_SIZE
shallow = ShallowTwoLayerNet(in_dim=shallow_in_dim, hidden_dim=64,
num_classes=num_classes)
cfg_shallow = OptimConfig(lr=1e-3, batch_size=1)

histories["P1A_Shallow"] = {"train_loss": [], "test_acc": []}
res_1a = run_experiment(
    deliverable_tag="Part 1A (Difficulty Check)",
    title="Shallow 2-layer MLP (Sigmoid) with SGD, batch=1",
    model=shallow,
    train_loader=train_b1,
    test_loader=test_b1,
    num_classes=num_classes,
    activation=act_sigmoid,
    cfg=cfg_shallow,
    epochs=EPOCHS_P1A,
    history=histories["P1A_Shallow"]
)

add_result_row(
    results,
    deliverable="1A",
    experiment="Shallow MLP",
    activation_name="sigmoid",
    batch_size=1,
    lr=cfg_shallow.lr,
    momentum_alpha=cfg_shallow.momentum_alpha,
    weight_decay_beta=cfg_shallow.weight_decay_beta,
    metrics=res_1a["final_metrics"]
)

# Print the “difficulty check” conclusion (report-ready)
subbanner("Difficulty Check Conclusion")
acc1a = res_1a["final_metrics"]["accuracy"]
print_kv([
    ("Shallow model accuracy", fmt_pct(acc1a)),
    ("Threshold", "50.00%"),
    ("Result", difficulty_pass_fail(acc1a, 0.50)),
```

```
    ])

    # ========================================================
    # PART 1B — Baseline Deep Network (Sigmoid, SGD, batch=1)
    # ========================================================
    baseline = BaselineDeepNet(image_size=IMAGE_SIZE, num_classes=num_classes,
hidden_dim=256)
    cfg_base = OptimConfig(lr=1e-3, batch_size=1)

    histories["P1B_BaselineSigmoid"] = {"train_loss": [], "test_acc": []}
    res_1b = run_experiment(
        deliverable_tag="Part 1B (Baseline Deep Model)",
        title="Baseline Deep Net (Sigmoid) with SGD, batch=1",
        model=baseline,
        train_loader=train_b1,
        test_loader=test_b1,
        num_classes=num_classes,
        activation=act_sigmoid,
        cfg=cfg_base,
        epochs=EPOCHS_P1B,
        history=histories["P1B_BaselineSigmoid"]
    )

    add_result_row(
        results,
        deliverable="1B",
        experiment="Baseline Deep Net",
        activation_name="sigmoid",
        batch_size=1,
        lr=cfg_base.lr,
        momentum_alpha=cfg_base.momentum_alpha,
        weight_decay_beta=cfg_base.weight_decay_beta,
        metrics=res_1b["final_metrics"]
    )

    # Random chance note (helpful for analysis)
    subbanner("Baseline vs Random Chance (context for analysis)")
    rc = random_chance_accuracy(num_classes)
    print_kv([
```

```
    ("#Classes", str(num_classes)),
    ("Random chance accuracy", fmt_pct(rc)),
    ("Baseline accuracy", fmt_pct(res_1b["final_metrics"]["accuracy"])),
])

# ==========================================================
# PART 2A / 2B — Two Activation Experiments (one per run)
# ==========================================================
# 2A: tanh
baseline_tanh = BaselineDeepNet(IMAGE_SIZE, num_classes, hidden_dim=256)
histories["P2A_Tanh_B1"] = {"train_loss": [], "test_acc": []}
res_2a = run_experiment(
    deliverable_tag="Part 2A (Activation Experiment 1)",
    title="Baseline + tanh with SGD, batch=1",
    model=baseline_tanh,
    train_loader=train_b1,
    test_loader=test_b1,
    num_classes=num_classes,
    activation=act_tanh,
    cfg=cfg_base,
    epochs=EPOCHS_P2,
    history=histories["P2A_Tanh_B1"]
)

add_result_row(
    results,
    deliverable="2A",
    experiment="Baseline + tanh",
    activation_name="tanh",
    batch_size=1,
    lr=cfg_base.lr,
    momentum_alpha=cfg_base.momentum_alpha,
    weight_decay_beta=cfg_base.weight_decay_beta,
    metrics=res_2a["final_metrics"]
)

# 2B: leaky relu
baseline_lrelu = BaselineDeepNet(IMAGE_SIZE, num_classes, hidden_dim=256)
histories["P2B_LeakyReLU_B1"] = {"train_loss": [], "test_acc": []}
```

```
res_2b = run_experiment(
    deliverable_tag="Part 2B (Activation Experiment 2)",
    title="Baseline + LeakyReLU with SGD, batch=1",
    model=baseline_lrelu,
    train_loader=train_b1,
    test_loader=test_b1,
    num_classes=num_classes,
    activation=act_leaky_relu_01,
    cfg=cfg_base,
    epochs=EPOCHS_P2,
    history=histories["P2B_LeakyReLU_B1"]
)

add_result_row(
    results,
    deliverable="2B",
    experiment="Baseline + LeakyReLU",
    activation_name="leaky_relu(0.1)",
    batch_size=1,
    lr=cfg_base.lr,
    momentum_alpha=cfg_base.momentum_alpha,
    weight_decay_beta=cfg_base.weight_decay_beta,
    metrics=res_2b["final_metrics"]
)

# Choose best activation based on final accuracy
best_act_name, best_act = ("tanh", act_tanh) if res_2a["final_metrics"]["accuracy"] >=
res_2b["final_metrics"]["accuracy"] else ("leaky_relu(0.1)", act_leaky_relu_01)
subbanner("Activation Winner (used for next deliverables)")
print_kv([
    ("Winner", best_act_name),
    ("Reason", "Highest final test accuracy among the two activation experiments"),
])

# ==========================================================
# PART 2C — Mini-batch SGD with best activation
# ==========================================================
batch_sizes = [8, 32, 64]
mb_runs = []
```

```
for b in batch_sizes:
    train_b, test_b = loaders_for_b(b)
    m = BaselineDeepNet(IMAGE_SIZE, num_classes, hidden_dim=256)
    cfg_mb = OptimConfig(lr=1e-3, batch_size=b)

    res_mb = run_experiment(
        deliverable_tag="Part 2C (Mini-batch SGD)",
        title=f"Mini-batch SGD (b={b}) using best activation ({best_act_name})",
        model=m,
        train_loader=train_b,
        test_loader=test_b,
        num_classes=num_classes,
        activation=best_act,
        cfg=cfg_mb,
        epochs=EPOCHS_P2
    )
    mb_runs.append((b, res_mb))

    add_result_row(
        results,
        deliverable="2C",
        experiment=f"Mini-batch SGD b={b}",
        activation_name=best_act_name,
        batch_size=b,
        lr=cfg_mb.lr,
        momentum_alpha=cfg_mb.momentum_alpha,
        weight_decay_beta=cfg_mb.weight_decay_beta,
        metrics=res_mb["final_metrics"]
    )

mb_runs.sort(key=lambda x: x[1]["final_metrics"]["accuracy"], reverse=True)
best_b = mb_runs[0][0]
subbanner("Best Batch Size (used for momentum + Part 3)")
print_kv([("Best batch size", str(best_b))])

# ========================================================
# PART 2D — Mini-batch SGD + Momentum
# ========================================================
```

```
train_bestb, test_bestb = loaders_for_b(best_b)
alphas = [0.5, 0.9, 0.95]
mom_runs = []

for a in alphas:
    m = BaselineDeepNet(IMAGE_SIZE, num_classes, hidden_dim=256)
    cfg_mom = OptimConfig(lr=1e-3, batch_size=best_b, momentum_alpha=a)

    res_mom = run_experiment(
        deliverable_tag="Part 2D (Momentum)",
        title=f"Mini-batch SGD + Momentum (alpha={a}) using best activation ({best_act_name})",
        model=m,
        train_loader=train_bestb,
        test_loader=test_bestb,
        num_classes=num_classes,
        activation=best_act,
        cfg=cfg_mom,
        epochs=EPOCHS_P2
    )
    mom_runs.append((a, res_mom))

    add_result_row(
        results,
        deliverable="2D",
        experiment=f"Momentum alpha={a}",
        activation_name=best_act_name,
        batch_size=best_b,
        lr=cfg_mom.lr,
        momentum_alpha=cfg_mom.momentum_alpha,
        weight_decay_beta=cfg_mom.weight_decay_beta,
        metrics=res_mom["final_metrics"]
    )

mom_runs.sort(key=lambda x: x[1]["final_metrics"]["accuracy"], reverse=True)
best_alpha = mom_runs[0][0]
subbanner("Best Momentum Alpha (used for Part 3)")
print_kv([("Best alpha", str(best_alpha))])

# ========================================================
```

```
# PART 3A — Extended +10 layers, record epoch-1 gradients
# ========================================================
ext = ExtendedDeepNetWithSkips(IMAGE_SIZE, num_classes, hidden_dim=256,
extra_layers=10)
cfg_ext = OptimConfig(lr=1e-3, batch_size=best_b, momentum_alpha=best_alpha)

histories["P3A_Extended_NoSkips"] = {"train_loss": [], "test_acc": []}
res_3a = run_experiment(
    deliverable_tag="Part 3A (Extended Model)",
    title="Extended model (+10 layers), no skips — record epoch-1 gradients",
    model=ext,
    train_loader=train_bestb,
    test_loader=test_bestb,
    num_classes=num_classes,
    activation=best_act,
    cfg=cfg_ext,
    epochs=EPOCHS_P3,
    record_grad_epoch1=True,
    skip_config=None,
    history=histories["P3A_Extended_NoSkips"]
)

add_result_row(
    results,
    deliverable="3A",
    experiment="Extended (+10), no skips",
    activation_name=best_act_name,
    batch_size=best_b,
    lr=cfg_ext.lr,
    momentum_alpha=cfg_ext.momentum_alpha,
    weight_decay_beta=cfg_ext.weight_decay_beta,
    metrics=res_3a["final_metrics"]
)

if res_3a["epoch1_grad_layer_report"] is not None:
    print_layerwise_grad_report("Part 3A Gradient Report (epoch 1)",
res_3a["epoch1_grad_layer_report"])

# ========================================================
```

```
# PART 3B / 3C — Two skip configs (3 skips each)
# ==========================================================
skip_config_1 = [(0, 2), (2, 5), (5, 10)]
skip_config_2 = [(0, 1), (1, 3), (3, 6)]

s1 = ExtendedDeepNetWithSkips(IMAGE_SIZE, num_classes, hidden_dim=256, extra_layers=10)
histories["P3B_Skip1"] = {"train_loss": [], "test_acc": []}
res_3b = run_experiment(
    deliverable_tag="Part 3B (Skip Config 1)",
    title=f"Skip config 1: {skip_config_1} — record epoch-1 gradients",
    model=s1,
    train_loader=train_bestb,
    test_loader=test_bestb,
    num_classes=num_classes,
    activation=best_act,
    cfg=cfg_ext,
    epochs=EPOCHS_P3,
    record_grad_epoch1=True,
    skip_config=skip_config_1,
    history=histories["P3B_Skip1"]
)

add_result_row(
    results,
    deliverable="3B",
    experiment="Skip config 1",
    activation_name=best_act_name,
    batch_size=best_b,
    lr=cfg_ext.lr,
    momentum_alpha=cfg_ext.momentum_alpha,
    weight_decay_beta=cfg_ext.weight_decay_beta,
    metrics=res_3b["final_metrics"]
)

if res_3b["epoch1_grad_layer_report"] is not None:
    print_layerwise_grad_report("Part 3B Gradient Report (epoch 1)",
res_3b["epoch1_grad_layer_report"])

s2 = ExtendedDeepNetWithSkips(IMAGE_SIZE, num_classes, hidden_dim=256, extra_layers=10)
```

```
    histories["P3C_Skip2"] = {"train_loss": [], "test_acc": []}
    res_3c = run_experiment(
        deliverable_tag="Part 3C (Skip Config 2)",
        title=f"Skip config 2: {skip_config_2} — record epoch-1 gradients",
        model=s2,
        train_loader=train_bestb,
        test_loader=test_bestb,
        num_classes=num_classes,
        activation=best_act,
        cfg=cfg_ext,
        epochs=EPOCHS_P3,
        record_grad_epoch1=True,
        skip_config=skip_config_2,
        history=histories["P3C_Skip2"]
    )

    add_result_row(
        results,
        deliverable="3C",
        experiment="Skip config 2",
        activation_name=best_act_name,
        batch_size=best_b,
        lr=cfg_ext.lr,
        momentum_alpha=cfg_ext.momentum_alpha,
        weight_decay_beta=cfg_ext.weight_decay_beta,
        metrics=res_3c["final_metrics"]
    )

    if res_3c["epoch1_grad_layer_report"] is not None:
        print_layerwise_grad_report("Part 3C Gradient Report (epoch 1)",
res_3c["epoch1_grad_layer_report"])

    # ==========================================================
    # FINAL: print summary table
# ==========================================================
    print_results_summary_table(results)

    # ==========================================================
    # ==========================================================
```

```
    # Learning curves (select only key runs)
    plot_learning_curves({
        "P1B Baseline Sigmoid": histories["P1B_BaselineSigmoid"],
        "P2A Baseline + tanh": histories["P2A_Tanh_B1"],
        "P3A Extended no skips": histories["P3A_Extended_NoSkips"],
        "P3B Skip config 1": histories["P3B_Skip1"],
    })

    # Gradient plots (top-k layers)
    if res_3a["epoch1_grad_layer_report"]:
        plot_top_gradients("Part 3A: Epoch-1 Gradients (No Skips)",
res_3a["epoch1_grad_layer_report"])
    if res_3b["epoch1_grad_layer_report"]:
        plot_top_gradients("Part 3B: Epoch-1 Gradients (Skip Config 1)",
res_3b["epoch1_grad_layer_report"])
    if res_3c["epoch1_grad_layer_report"]:
        plot_top_gradients("Part 3C: Epoch-1 Gradients (Skip Config 2)",
res_3c["epoch1_grad_layer_report"])

    banner("DONE")

if __name__ == "__main__":
    main()
```